

调度策略

在到达调度时机后，处理器调度器根据具体的调度策略做出调度决策，然后系统根据该决策进行任务切换。这里的调度决策包括：①从运行队列中选择下一个运行的任务；②决定该任务下一次允许运行的时间，即时间片。根据调度策略的不同，在相同情况下做出的决策也会不尽相同。回顾代码片段 7.1 中的调度相关函数，它们实际上是操作系统的处理器调度所提供的接口。操作系统可以通过为这些接口提供不同的实现，允许用户选择不同的调度策略，达到机制与策略分离的效果。

在针对处理器调度的研究中，对于调度策略的研究一直是主旋律。这是因为没有一种完美的调度策略可以适应所有的场景。在设计调度策略的时候，需要考虑以下两个问题。

- 第一个问题是应该选择什么样的调度指标（What）？操作系统使用调度器的目的是通过协调任务对 CPU 的使用，进而使任务的执行在某一方面达到用户的预期，或者说达到用户指定的调度指标。那么有哪些指标？在不同种类的系统中应该考虑何种指标？这些都是开发者在设计实现调度器时需要考虑的因素，具体会在 7.2 节进行详细介绍。
- 第二个问题是应该如何选择符合预期的调度决策（How）？在确定当前系统需要考虑的调度指标后，我们需要进一步通过适当的调度策略做出合理的决策。应该如何选择合理的调度策略？这需要开发者对于不同调度策略有着清晰的认识，具体会在 7.3 ~ 7.5 节进行详细介绍。

小知识：空闲进程

细心的读者可能会问，如果运行队列中没有可执行的任务，那么 CPU 核心应该执行什么代码呢？实际上，当运行队列为空时，CPU 核心一般会执行一段预设的“无意义”循环代码。为了处理器调度设计的统一性，大部分操作系统都倾向于让上述“无意义”的代码由一类特殊的进程执行，此类进程被称为**空闲进程（Idle Process）**。这段“无意义”的代码一般会让 CPU 核心进入低功耗模式。以 ChCore 为例，在 ARM 架构下，空闲进程会执行 wfi 指令^①，直到中断发生，当前 CPU 才会重新正常工作。

练习

7.1 既然低功耗模式可以节省能耗，那么操作系统是否应该让 CPU 核心一有机会就进入低功耗模式呢？

① 该指令全名为 wait for interrupt。执行该指令的 CPU 会停止执行代码并进入低功耗模式，直到该 CPU 接收到中断为止。

7.2 处理器调度指标

本节主要知识点

- 常见调度指标有哪些？
- 互相冲突的调度指标有哪些？

本节将介绍处理器调度指标，这些指标被用于评判调度策略的优劣，具体的调度策略会在后续章节讲解。需要注意的是，在评判一个调度策略的优劣时，一定是评判该调度策略在特定场景下的优劣。这是因为计算机的应用场景复杂多变，用户对于不同场景下的任务执行会有不同的预期，因此会有不同的调度指标来指导调度策略的选择。本节将介绍下列主要的调度指标：平均周转时间、平均响应时间、实时性、资源利用率、公平性和调度机制的开销。

平均周转时间

在计算机处理的任务中，有一类任务被称为批处理任务，比如机器学习的训练、复杂的科学计算等，它们在计算机上执行时无须与用户交互，其目标就是尽快完成。这类任务的主要调度指标是周转时间（Turnaround Time），即任务在系统中总共花费的时间。由于任务只要在系统中就会消耗资源，所以任务的周转时间越小越好。其计算公式是任务的完成时间点（Completion Time）与到达时间点（Arrival Time）之差^①：

$$T_{\text{周转}} = t_{\text{完成}} - t_{\text{到达}}$$

其中，到达时间点是任务在系统中被创建并开始等待处理器执行的时间点，完成时间点是任务结束执行并在系统中被销毁的时间点。

实际上，一个任务的周转时间还是它的等待时间（Waiting Time）与运行时间（Burst Time）之和：

$$T_{\text{周转}} = T_{\text{等待}} + T_{\text{运行}}$$

其中，等待时间是任务在系统中等待被执行所花费的时间，运行时间是执行任务所需的时间。一个任务的运行时间越长，所消耗的处理器的资源越多。根据运行时间的相对长短，任务可以被称为长任务或短任务。

在某些情况下，任务的运行时间是大致不变的，因此任务的周转时间越长，则代表任务花费了越多的时间在等待执行上。系统中所有任务的平均周转时间是一个经典的调度指标，用于衡量任务在系统中等待执行的情况。该指标越小，代表系统中全部任务花费在等待上的时间越少。

平均响应时间

随着计算机的应用场景越来越多，人们开始需要使用计算机执行一些交互式任

① 在本章的公式中， T 表示时间段， t 表示时间点。

务,例如文字处理、游戏。对于这类交互式任务来说,它们需要在执行过程中响应用户操作,因此周转时间并不是最重要的调度指标。这类任务的主要调度指标是响应时间(Response Time),即任务从被创建到第一次被处理器执行所需的时间。显然,用户希望自己的请求(例如键盘、鼠标输入)能够被及时响应,因此任务的响应时间越短越好。其计算公式是任务的第一次执行时间点与到达时间点之差:

$$T_{\text{响应}} = t_{\text{第一次执行}} - t_{\text{到达}}$$

顾名思义,第一次执行时间点是任务第一次被CPU执行的时间点。显然,该时间点越早,则任务可以越快响应用户的请求。系统中所有任务的平均响应时间用于衡量任务响应用户是否及时,同样是一个经典的调度指标。

实时性

操作系统还要处理有截止时间(Deadline)要求的实时任务。例如车载系统中的距离探测器会定时检测汽车与外部物体的距离,如果当前车速过快且距离过近,则车载系统会强制刹车。探测任务的实时性要求在限定时间内判断当前车速是否会导致碰撞,从而避免严重的安全事故。又如视频的画面渲染,每一帧画面需要在截止时间内完成进而保证视频的帧率,否则会造成卡顿。在系统保证实时任务执行结果正确的同时,调度还必须保证实时任务的实时性,即任务在截止时间内完成。

资源利用率

在多核场景下存在多个CPU核心,系统应该尽可能保证每个核心都在执行任务,进而提高处理器资源的利用率。除了处理器资源外,操作系统同时负责管理其他资源,例如磁盘、网络、GPU等。相对地,任务也可能访问多种资源。如果调度器在调度任务时仅仅考虑处理器资源的使用情况,虽然处理器资源会被充分利用,但系统整体的资源利用率可能会很低,系统性能无法达到最优。因此,调度策略应尽可能保证系统资源被充分利用。

公平性

在通常情况下,应保证每个任务都有被执行的可能。然而,某些调度策略在倾向于调度一类任务时,可能导致另一类任务一直无法被调度,处于饥饿(Starvation)状态。上述情况对于无法被调度的任务是不公平的。因此,调度策略应尽可能保证调度的公平性,即每个任务都有机会被执行。

调度机制的开销

除了满足上述常见的调度指标外,调度机制本身也会为系统整体带来开销。调度机制的开销可以分为两部分,一部分是做出决策的开销,另一部分则是任务上下文切换的开销。

调度器为了做出调度决策,需要访问任务信息并进行计算与决策,这一定会带来性能的开销。需要注意的是,针对相同的调度策略,不同调度器实现的计算复杂度可能会不同。假设系统中有若干任务,一个调度器实现的复杂度为 $O(N)$,即需要遍历所有任务才能做出决策。而另一个调度器实现的复杂度为 $O(1)$,即常数时间就能做出决策。显然,随着系统处理的并发任务数量的增长,后者做出一次决策的开销会比前者少很多。

另一方面，处理器在任务间切换也会带来开销，当任务切换频繁时，其开销是无法忽略的。系统使用调度器的目的是优化任务的执行，而非引入新的性能瓶颈。因此，调度器应尽可能高效地实现，而调度策略也要控制任务调度的频率，避免调度机制的过大开销。

练习

7.2 根据所学的知识，请列举出操作系统中任务上下文切换可能导致的调度开销。

7.3 你是否还能想到其他调度指标？

在了解了上述调度指标后，不难发现许多指标之间存在冲突与权衡，这一现象也说明了选取合适的调度指标与策略绝非易事。例如，大部分情况下平均响应时间与调度机制开销是相互冲突的。如果系统希望减少任务的平均响应时间，就需要减少任务的时间片。但这反而导致处理器单位时间内在多个任务间进行频繁切换，放大了任务切换的开销。又如，大部分情况下实时性和公平性之间存在权衡。在系统中，某些实时任务为了在截止时间前完成，必须优先于其他任务执行，这可能会导致其他任务长时间无法执行，产生公平性问题。

7.3 经典调度策略

本节主要知识点

□ 有哪些经典的调度策略？

□ 不同的经典调度策略存在哪些优劣？

为了满足复杂多变的调度指标，多种多样的调度策略应运而生，在处理器调度发展的历史上留下了浓墨重彩的一笔。本章将通过经典调度策略（7.3节）、优先级调度策略（7.4节）和公平共享调度策略（7.5节），带领读者逐步构建起对处理器调度策略的理解。阅读完后续章节，相信读者可以具备初步的调度策略设计能力，满足一些简单的调度指标。

本节将从一些直观且经典的调度策略开始讲解，例如先到先得、最短任务优先、最短完成时间优先和时间片轮转等。了解这些经典的调度策略有助于理解更为复杂的调度策略背后的设计权衡，以及理解当下系统调度器的设计。

7.3.1 先到先得

在调度策略中，非常容易想到的一种策略是先到先得（First Come First Served, FCFS）策略。这一类策略也被称为先进先出（First In First Out, FIFO）策略。“先来后到”经常体现在日常生活中，在不同场合人们会自发地排队，比如买票、在银行办理业务等，都